ASIA PACIFIC UNIVERSITY OF TECHNOLOGY & INNOVATION

CT013-3-3-CSS INDIVIDUAL ASSIGNMENT REPORT

TITLE	ATTENDANCE MANAGEMENT SYSTEM
NAME & STUDENT ID	NOOR UR RASHID (TP066720)
INTAKE	APD4F2601CE
LECTURER	DR. KAMALAKANNAN MACHAP
Date	27/03/2026

Table of Contents

List of Figures 4

1.0 Introduction 6

1.1 Overview of the Chosen Web-Application 6

1.2 Pages and Functionalities 7

1.2.1 Login Page 7

1.2.2 Admin Dashboard Page 8

1.2.3 Create Teacher Page 9

1.2.4 Teacher Account Created Email 10

1.2.5 Teacher Login Change Password Page 11

1.2.6 Teacher Dashboard 12

1.2.7 Teacher Mark Attendance Page 13

1.2.8 Create Student Page 14

1.2.9 Student Account Created Email 15

1.2.10 Student Login Change Password Page 15

1.2.11 Student Dashboard 16

1.2.12 View Attendance 17

1.2.13 View All Teacher & Student Page 18

1.2.14 Create Subject Page 19

1.2.15 Enroll Student Page 21

1.2.16 View All Subject Attendance 22

1.2.17 Invalid Login 23

1.2.18 Captcha Login Verification 24

1.2.19 Account Locked or Verification Via Email 25

1.2.20 Verification by OTP Page 26

1.2.21 OTP Expiration and Resend New OTP 26
1.2.22 OTP Verified and Account Unlocked 27
1.3 Program Technical Backend 28
2.0 Security Issue Identification 30
2.1 Unauthorized Access to Restricted Pages 30
2.2 Brute Force Login Attempts 30
2.3 Automated Login Attacks (Bot Attacks) 31
2.4 Weak Password Usage 31
2.5 Account Recovery Security Risk 32
2.6 Password Storage Risk 33
3.0 Requirement Identification 34
3.1 Secure Password Hashing Using bcrypt 34
3.2 Login Attempt Limitation to Prevent Brute Force Attacks 34
3.3 CAPTCHA Verification to Prevent Automated Attacks 35
3.4 Role-Based Access Control 35
3.5 OTP-Based Account Recovery System 36
3.6 Password Strength Validation 36
4.0 Conclusion 37
5.0 References 38

List of Figures

Figure 1 Login Page 7
Figure 2 Admin Dashboard 8
Figure 3 Create Teacher Page 9
Figure 4 Teacher account created email 10
Figure 5 Teacher login password change requirement 11
Figure 6 Teacher Dashboard 12
Figure 7 Mark Attendance 13
Figure 8 Create Student Page 14
Figure 9 Student account created email 15
Figure 10 Student login password change requirement 15
Figure 11 Student Dashboard 16
Figure 12 View Attendance 17
Figure 13 Existing Teacher and Student list 18
Figure 14 Create Subject Page 20
Figure 15 Enroll Student 21
Figure 16 All Subject Attendance 22
Figure 17 Wronged Password Entered 23
Figure 18 Captcha Verification 24
Figure 19 Account Locked 25
Figure 20 OTP Verification 26
Figure 21 Resend OTP 27
Figure 22 OTP Verified 27
Figure 23 Role-Based Access Control 30
Figure 24 Brute Force Login Protection 30
Figure 25 CAPTCHA Protection Against Automated Attacks 31
Figure 26 Password Strength Validation 31
Figure 27 OTP Generation 32

Figure 28 OTP Verification 32
Figure 29 Secure Password Storage 33
Figure 30 Secure Password Hashing Using bcrypt 34
Figure 31 Login Attempt Limitation to Prevent Brute Force Attacks 34
Figure 32 CAPTCHA Verification to Prevent Automated Attacks 35
Figure 33 Role-Based Access Control 35
Figure 34 OTP-Based Account Recovery System 36
Figure 35 Password Strength Validation 36

1.0 Introduction

1.1 Overview of the Chosen Web-Application

The web application selected is Web-Based Attendance Management System (AMS) that has been created with the aim of digitalizing and automating the process of recording and managing student attendance in a learning institution. Conventional methods of attendance, usually time-consuming and subject to error, are substituted with a centralized and safe online platform. The system has been built on Flask framework in Python as the backend processing framework whereas the user interface is made with HTML and CSS to make the system easy to use and accessible. The storage of user credentials and attendance records in an organised and structured format is done using a relational database (SQLite).

Role based access control system has also been adopted, in which users are grouped into students and teachers with access rights being given respectively. Students are also permitted to enroll, log in safely, record their attendance, and see their individual attendance history. Access to student attendance data is availed to teachers in a special dashboard where they can monitor and control attendance data. Authentication and session management systems have been incorporated to make sure that unauthorized users cannot access the system. To provide an extra security, passwords are stored with the help of hashing, and to limit the chances of SQL injection attacks, the structured database queries are applied. The practical use of the concepts of web development and cybersecurity in the academic setting is proven through the implementation of this web application that allows achieving greater efficiency, higher data security, and minimization of human error, as well as access to the attendance data in real time.

1.2 Pages and Functionalities

1.2.1 Login Page

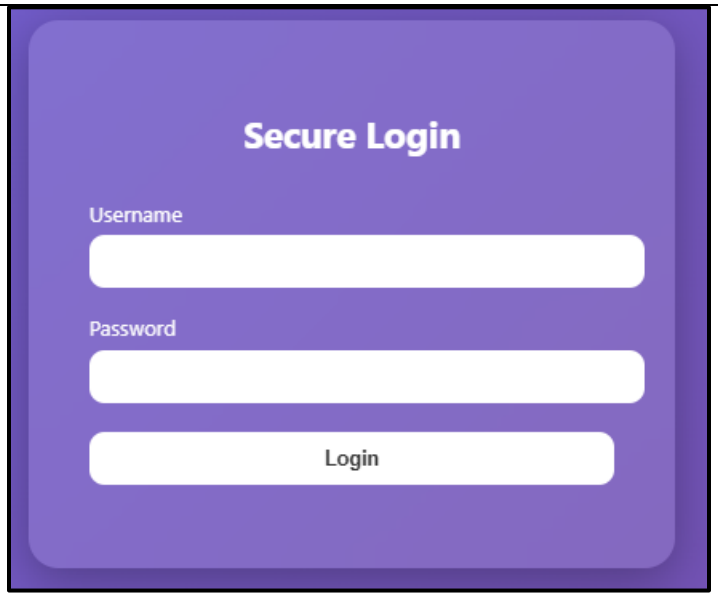


Figure 1 Login Page

The Attendances Management System access is carefully secured through the login page to enable registered users access it. The log-in form will prompt the user to provide their username and password to cover up their identity. The credentials typed are sent to the server where the password is checked with the hashed password, which is stored in the database through bcrypt. In case of the correct credentials, the user will be redirected to the proper dashboard depending on his or her role assigned, being an administrator, teacher, or student.

There are other security measures that are put in place to guard the process of logging-in. CAPTCHA verification follows three unsuccessful attempts at logging in so that the user is not the machine that is trying to log in to the network. When the number of unsuccessful attempts is five, the account is temporarily locked in five minutes, and a countdown timer is shown on the page. Users can also use email to unlock their account by verifying their identity by use of an OTP verification option. These mechanisms assist in increasing the security of the system in general.

1.2.2 Admin Dashboard Page

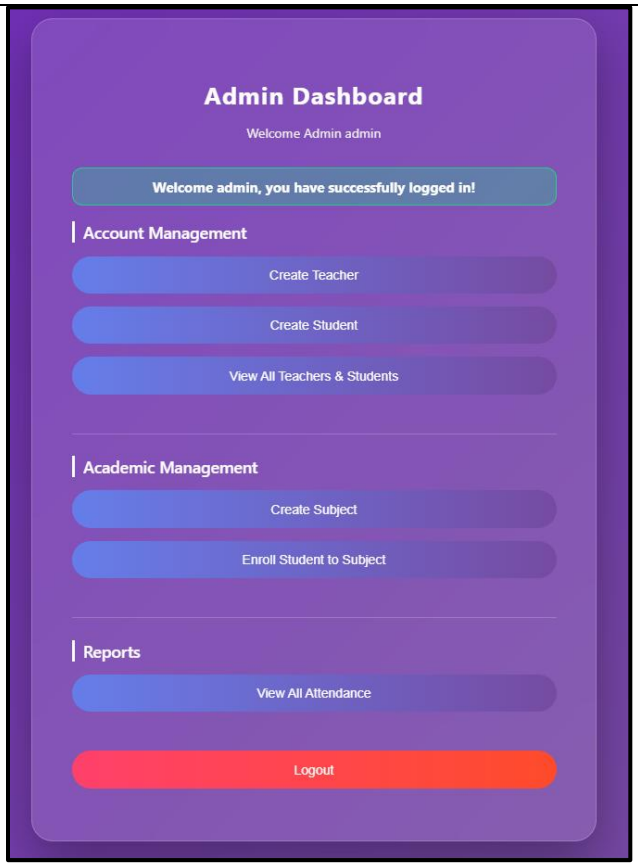


Figure 2 Admin Dashboard

Admin Dashboard is meant to give the administrative users access to the system management capabilities of the Attendance Management System. Once the administrator successfully logs in, he is redirected to this page where he or she can perform different administrative tasks. The dashboard shows a welcome message containing the username of the administrator and offers a number of options to perform with the system.

The administrator can use the dashboard to create teacher and student accounts, see all the registered users, create subjects and add students to subjects. Besides, the administrator has access to attendance reports where he or she can monitor attendance records in the system. Flash messages are also seen as a notification to the administrator about successful operations or messages about the system. The role-based access control is used to restrict access to this page so that only users with the administrator role can access the dashboard.

1.2.3 Create Teacher Page

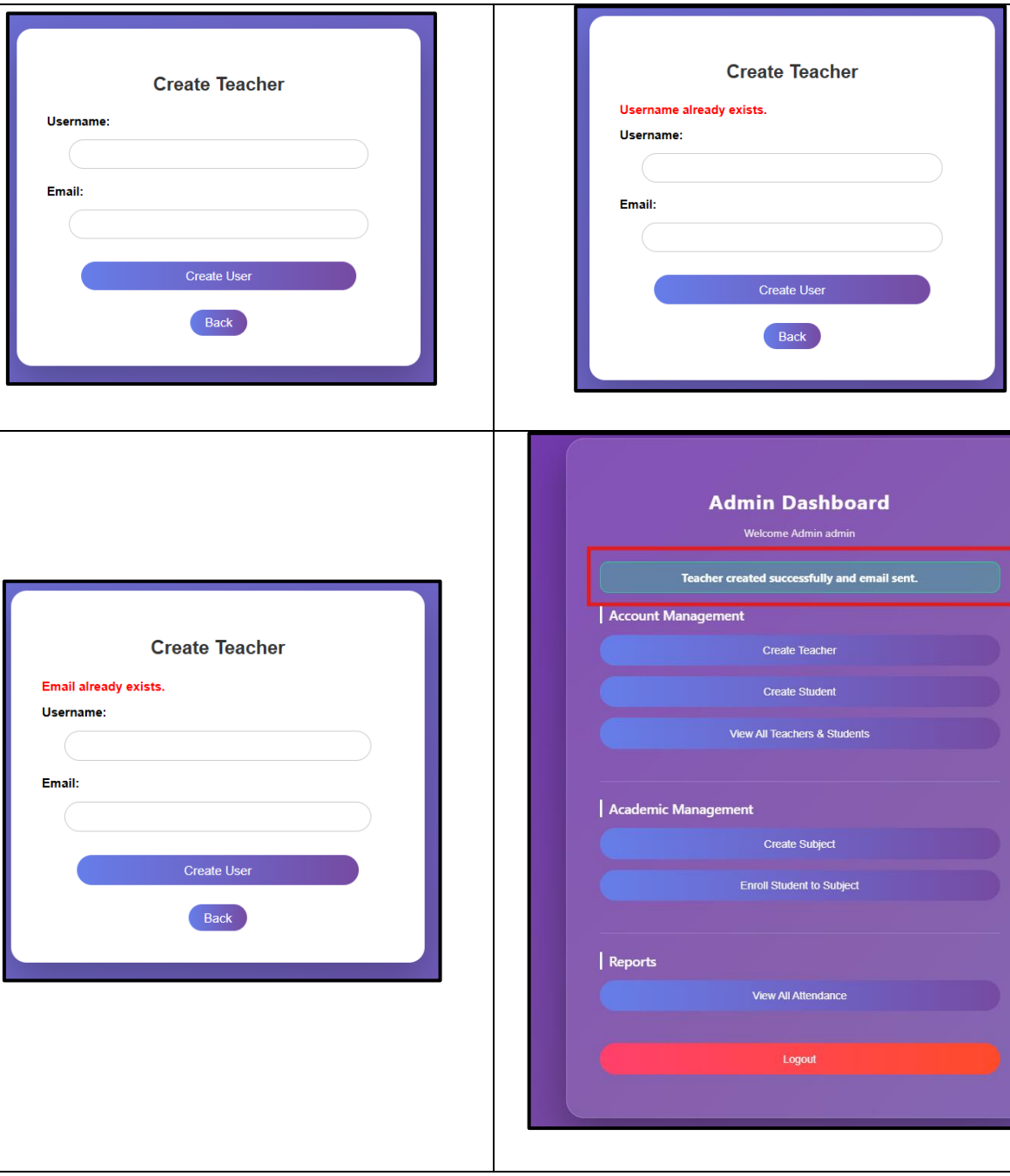


Figure 3 Create Teacher Page

The administrator creates new teacher accounts in the Attendance Management System on the Create Teacher page. There is a form given that the administrator is expected to fill in the username and email address of the teacher. Upon submission, the system checks the input and confirms that the username or email does not exist in the database in order to avert creation of similar accounts.

In case the information is valid, it is generated into a temporary password and hashed with bcrypt insecurely and then stored in the database. Another new teacher account is then added, and the role of the teacher is added on it. The temporary log in credentials are sent automatically to the teacher email address and the teacher has to change the password during the initial log in. This page is highly secured, meaning administrators are the only ones who can create accounts of teachers.

1.2.4 Teacher Account Created Email

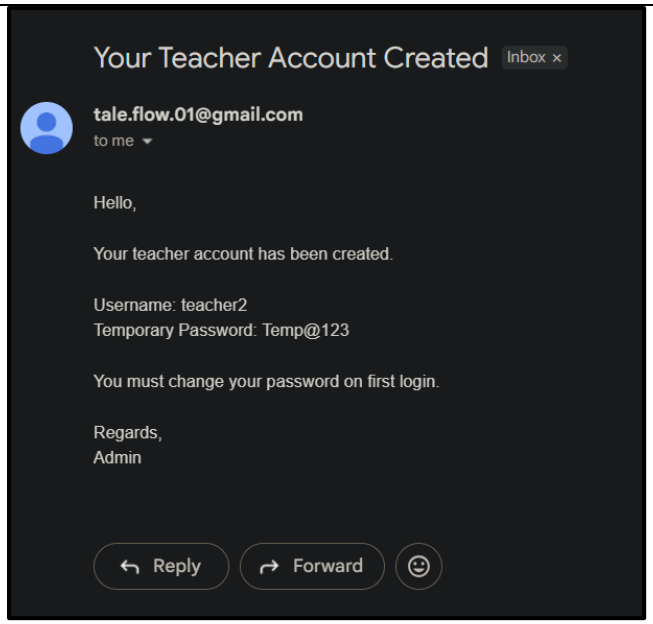


Figure 4 Teacher account created email

The email notification has been sent through the address tale.flow.01@gmail.com to notify the recipient that the new teacher account has been created. The username is mentioned in the body of the message as teacher2 and a temporary password is given as Temp@123 which would be required initially. It has a compulsory message that the user has to change his password on the first login. Lastly, communication ends with the signature of "Admin" as a sign that a system administrator finalized the setup.

1.2.5 Teacher Login Change Password Page

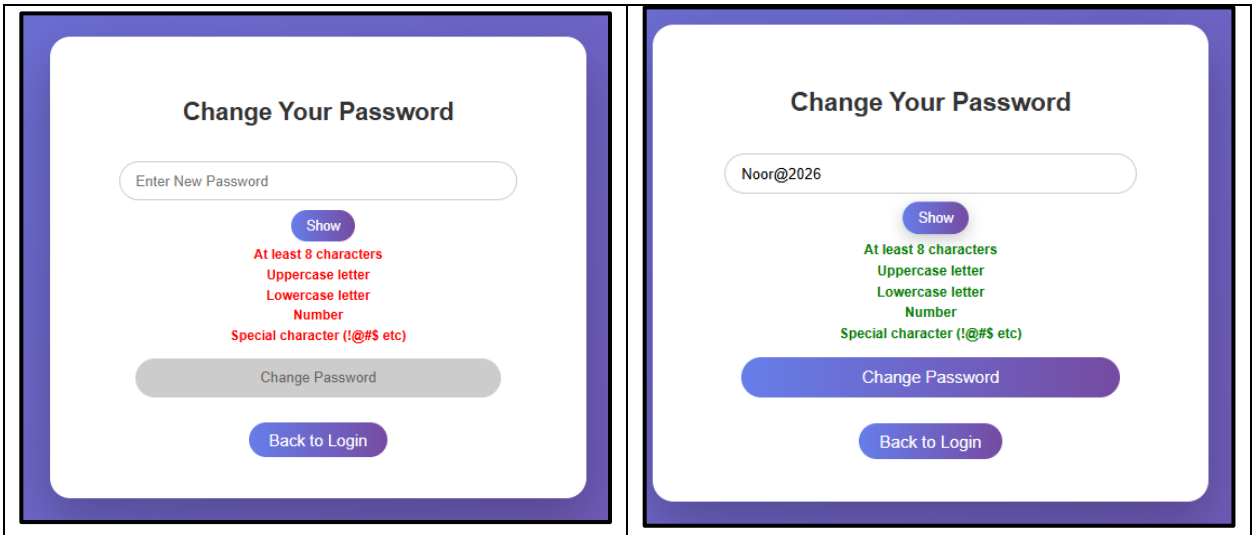


Figure 5 Teacher login password change requirement

When a teacher accesses the system with a temporary password given to them when the account was created, the Teacher Change Password page is shown. To provide security to the accounts, teachers must alter this temporary password prior to gaining entry to the system. In this page, the teacher is required to feed in a new password that meets the password security requirement of the system.

There is a password strength validation that can be used to create a strong password. The password has to have a minimum length of eight characters with the presence of both lower and upper letters, numbers, and special characters. These requirements are verified in real time by the system and only after that the password is submitted by the system when all the conditions are met. After the new password is entered, it is hashed with the help of the bcrypt algorithm and stored in the database. Once the password has been changed successfully, the teacher will be sent to the teacher dashboard.

1.2.6 Teacher Dashboard

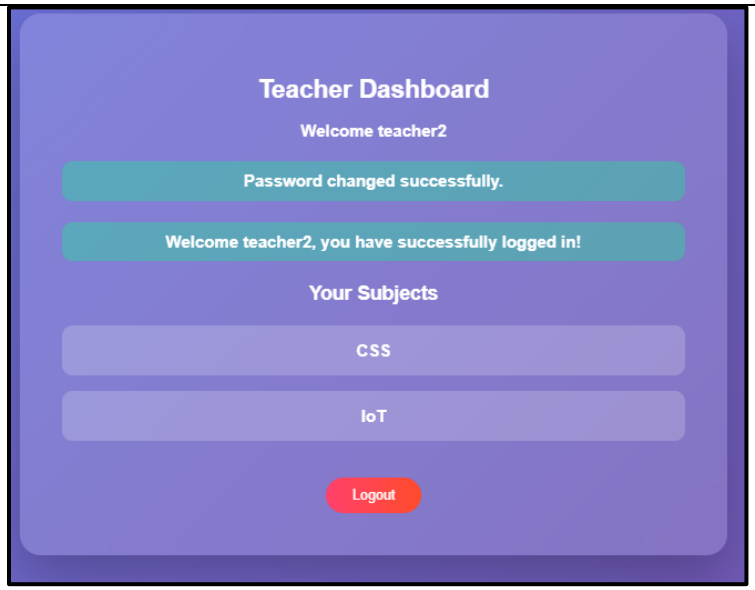
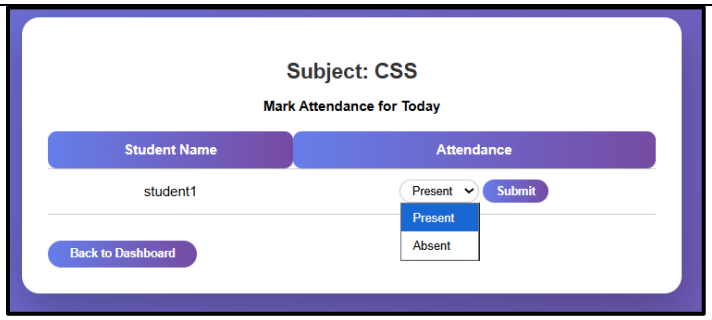


Figure 6 Teacher Dashboard

The Teacher Dashboard will enable teachers to have the access to the subjects they are in charge of in the Attendance Management System. Once the teachers have successfully logged in, they are redirected to this page which presents a welcome message that contains the name of the teacher. The dashboard pulls the list of the subjects that the teacher is assigned to, according to the enrolment data contained in the database.

All of the subjects will have a clickable section, which will enable the teacher to switch to the subject page, where he/she can carry out other activities related to attendance. There are also flash messages which show the teacher of successful login or system messages. Role-based access control is also used to restrict access to the Teacher Dashboard such that users with the role of teachers are only permitted to access it. There is also a logout system that lets the teachers leave the system in a secure manner.

1.2.7 Teacher Mark Attendance Page



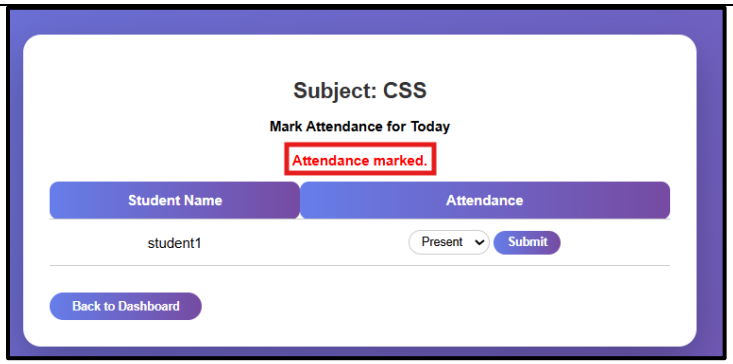


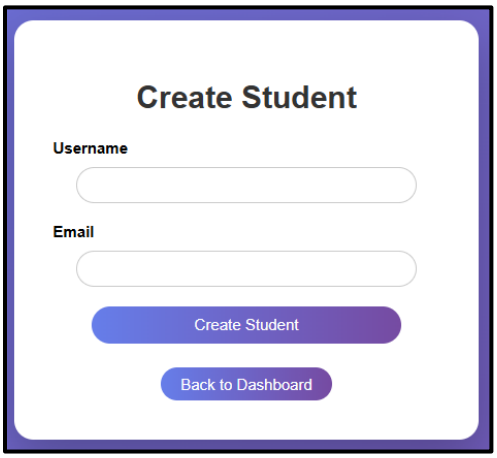
Figure 7 Mark Attendance

Teachers are supposed to use The Teacher Mark Attendance page to record student attendance in a particular subject. When a teacher chooses a subject in the Teacher Dashboard, the system retrieves the list of students enrolled in the subject using the records of teacher-student enrollment that is stored in the database. These students are then posted on the page in order to be taken note of by way of attendance.

Once the teacher enters the attendance, the system will be used to verify whether the student had been marked present on the given day. In case the attendance has already been marked, the notification of the same is shown to avoid repetition. In case no record is found, a new record is added into the database using the student ID, teacher ID, subject ID, date and attendance status. The process will make sure that every student only attends each subject once a day and records the attendance correctly.

This page can only be accessed by users who have the teacher role so that only qualified teachers can be given the chance to record the attendance of the subjects that they are assigned.

1.2.8 Create Student Page



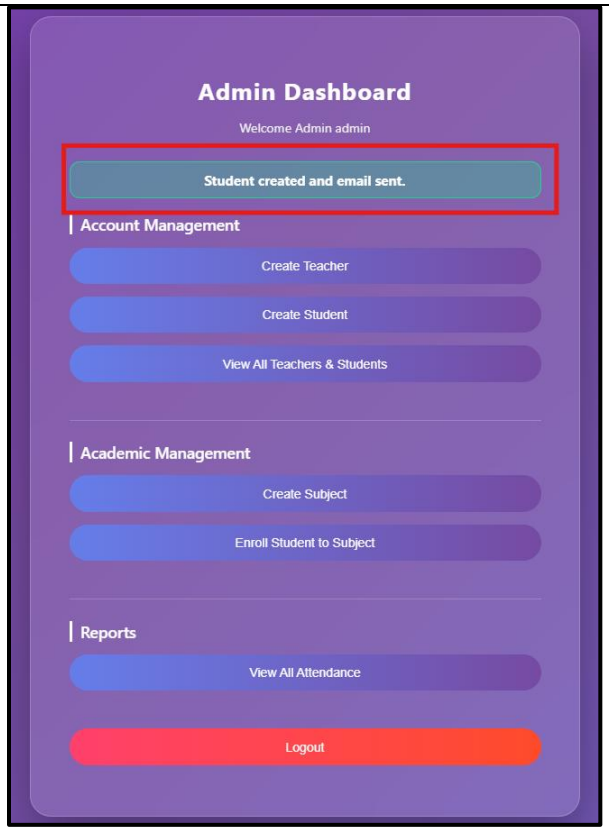


Figure 8 Create Student Page

The administrator creates new student accounts in the Attendance Management System on the Create Student page. This page is the one that can only be opened by the users with the administrator role to make sure that this action is performed only by the authorized users. It offers a form in which the administrator will be asked to write the name of the student and email address. On submission of the form the system checks the input and searches the database to confirm that the username and email is not an existing one.

Upon entering the information, a new student account will be created using a temporary password in case the information is valid. User credentials are protected by storing the password in the database after hashing it. Once the account has been successfully created, an email with the login information of the student and a temporary password is automatically dispatched to the email address given. To enhance account security, the student must change the password that he or she will use on the first login.

1.2.9 Student Account Created Email

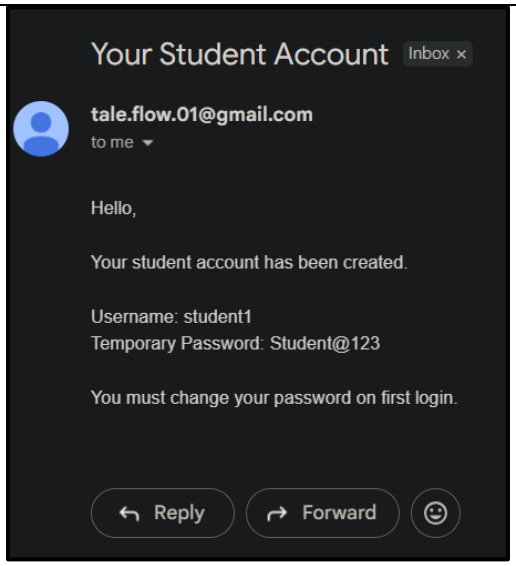


Figure 9 Student account created email

An email notification has been sent to the tale.flow.01@gmail.com to confirm that a student account is created. In the message, there is a username that is student1 and a temporary password which is given as Student@123. Included is a special command that the change in password has to be done by the user on the first login.

1.2.10 Student Login Change Password Page

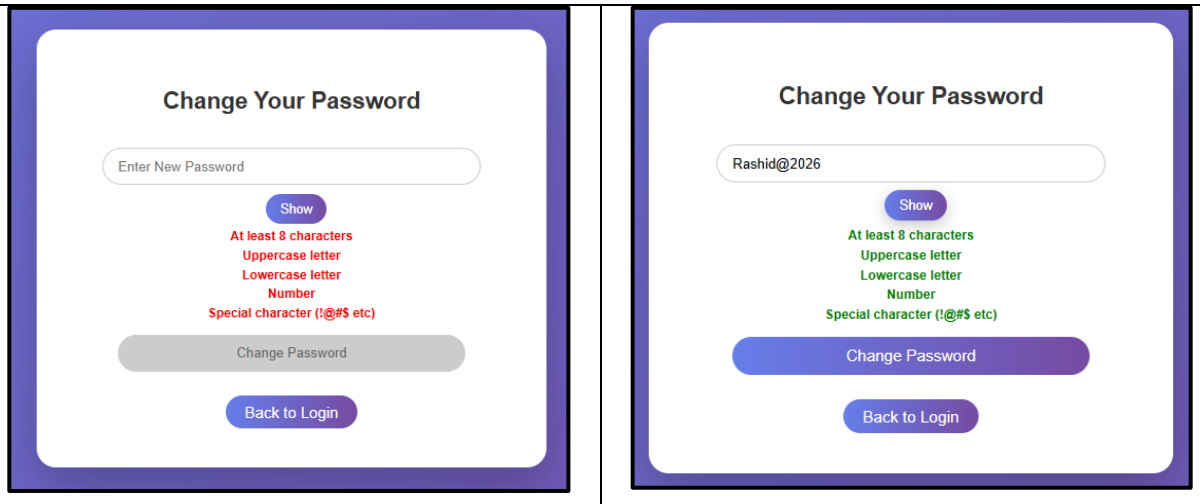


Figure 10 Student login password change requirement

Student Change Password page works in the same manner of the teacher password change process. Upon entering the system, a student applies a temporary password, which, in turn, demands the student to change the password in order to have the full access to the system. This is necessary to ensure that temporary credentials cannot be used over an extended period of time.

Students will have to draw a new password according to the established security policies, such as the required minimum length, use of capital letters, capitalized letters, numbers, and special characters. The password specifications are shown on the page and are verified prior to the page submission. After typing a valid password, the information is saved in the database after being encrypted with the use of the bcrypt hashing algorithm. Once the password change process has been successfully done, the student is redirected to the student dashboard.

1.2.11 Student Dashboard

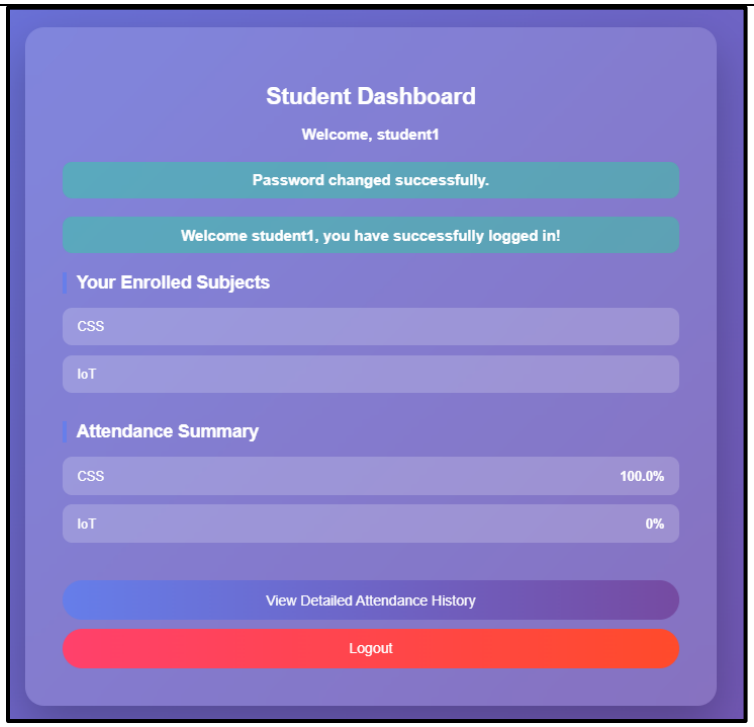


Figure 11 Student Dashboard

Upon logging into the system, the first page that a student will see is the Student Dashboard. It will provide a good picture of the courses taken and attendance by the student. Once a student enters this page, the system will first verify that the user logging in is a student; failure to which

he or she will be redirected into the login page. In the event that the process of logging in was successful, there is a flash message that greets the student. The dashboard will then retrieve all the subjects the student is enrolled in by making a query of the TeacherStudentEnrollment table and any attendance history of the student in the Attendance table.

The page shows two major sections, which include, Enrolled Subjects and Attendance Summary. The Enrolled Subjects is where each of the subjects that the student is enrolled in is displayed and in case the student is not enrolled in any of the subjects, the message is displayed. Under the attendance summary section, the system computes the percentage of attendance of each subject and further divides the number of present entries by the number of classes attended and shows the percentage. The students are also able to press a button which allows them to see the history of attendance in details in another page, or press the log out button. Such an arrangement gives the students a prompt and graphical overview of their current performance in terms of attendance.

1.2.12 View Attendance

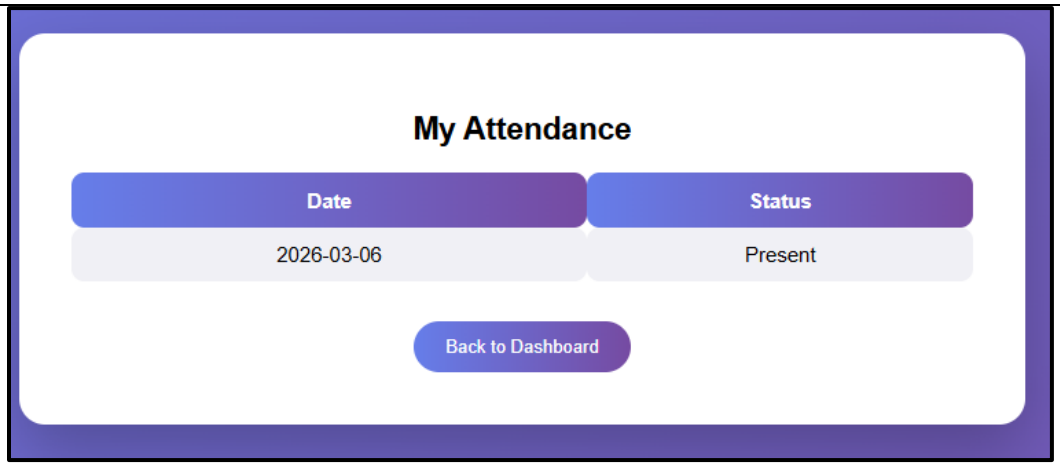


Figure 12 View Attendance

View Attendance page enables a student to view the history of attendance of all subjects in a detailed manner. Once a student clicks on this page, the backend will in the first instance, ensure that the user has a role of student and redirects any other person to the login page to ensure security. The system will then query the Attendance table to retrieve all the records that are of the student who is logged in (user_id = current_user.id). In every record, there will be the date of the class and the status of attendance which is typically either Present or Absent. The page also works out the

percentage attendance of each subject based on the number of classes that were marked as Present compared with the total number of classes that a subject has, however in this template the entries of individual attendance are presented solely in a table.

On the front end, the attendance records are presented in a table format in a clean manner. The table is arranged according to the date and status of the class and thus the students can easily go through their attendance history through time. In case there are no records, yet a message can be shown to the student, notifying that there are no records about attendance. It also has a button to go back to the Student Dashboard and the student is able to maneuver back to the general view of their subjects and attendance percentages. This one is a comprehensive analysis that is basically a breakdown of the page presented on the dashboard.

1.2.13 View All Teacher & Student Page

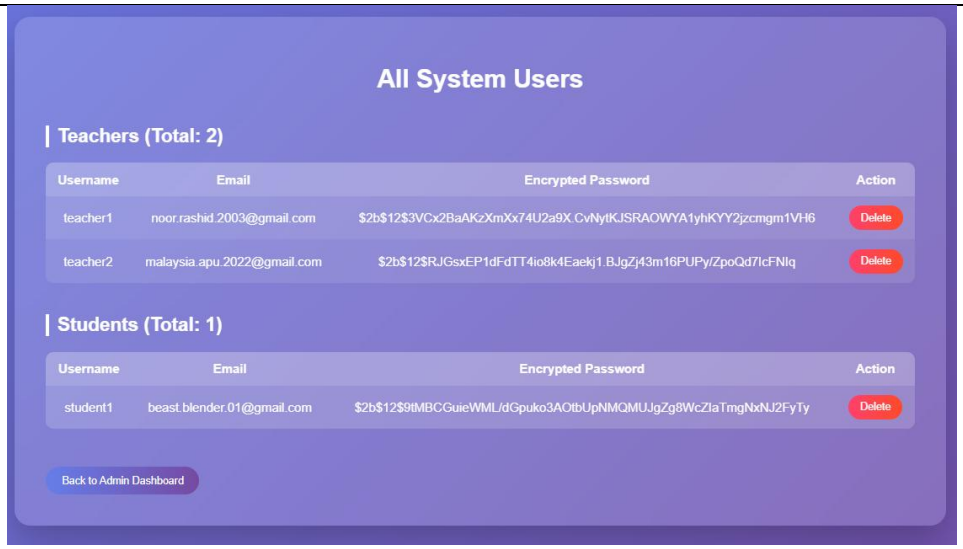


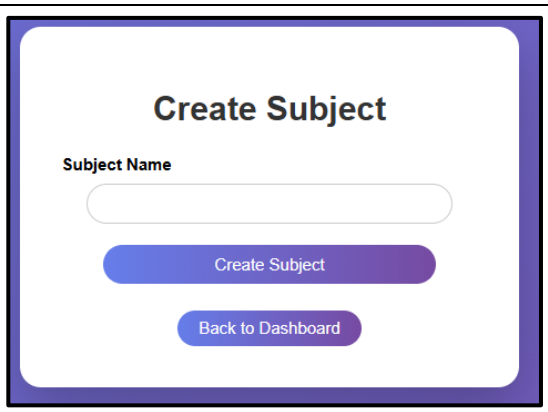
Figure 13 Existing Teacher and Student list

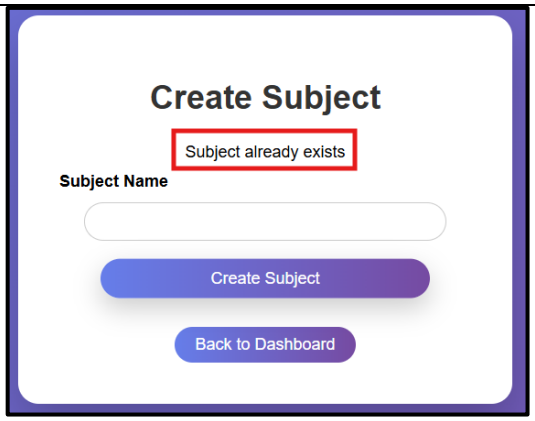
View users page is intended to enable the administrators to view all the users in the system divided into teachers and students. Once an admin accesses this page, the backend checks that the user who accesses this page is an admin and redirects the rest to the login page to avoid security breach. The system then uses the query to the User table to retrieve all teacher and student accounts individually and counts them to be used in the top of each section. The information on the page is arranged in a form of tables and indicates the username, email and ciphertext of the password of every user and provides the administrator with the comprehensive view of users on system. Flash

messages can be seen on the top with messages such as the success or error messages that occur in response to activities in this page.

Another thing that the page enables the admin to do is to delete a user. In each row, there is a Delete button that issues a POST request to the delete-user route where the backend makes a number of significant checks. It does not allow the admin to delete his account, all the attendance and enrollment records of the user are removed to keep the database intact and the user is deleted in the User table. Before the deletion, a confirmation pop-up is provided in order to prevent an accidental deletion. Once it has been deleted, a flash message confirms that it has been deleted either to the teacher or the student. Lastly, there is a button that takes one to the Back to Admin Dashboard to get back to the main administration dashboard easily. This architecture provides the administrators with complete control over user management and data consistency.

1.2.14 Create Subject Page





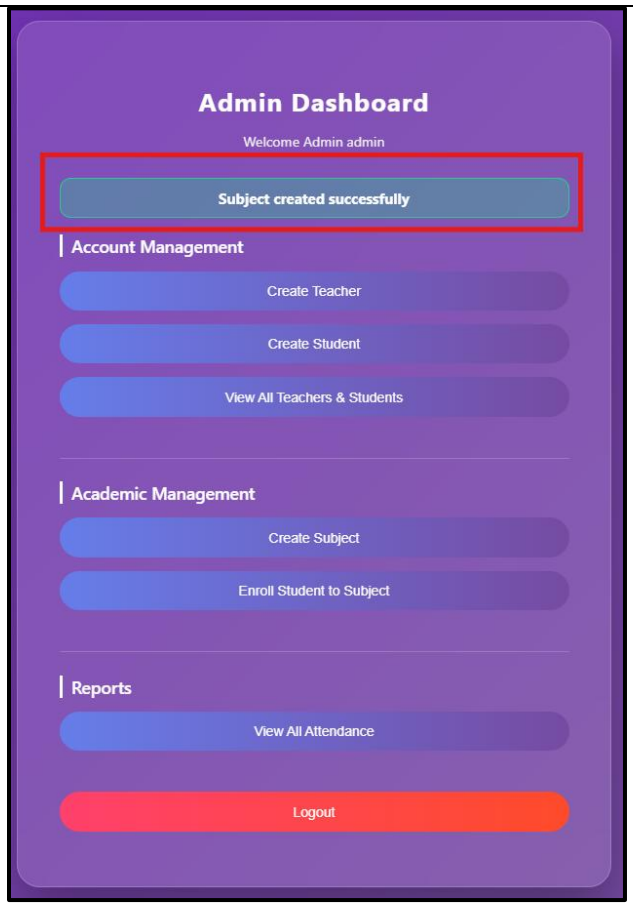


Figure 14 Create Subject Page

Create Subject page enables an administrator to create new subjects in the system. On opening this page, the system will first check that the user who has been logged in has an administrative role; otherwise, he or she will be redirected to the log-in page. The page has a basic form on which the admin can type the name of the subject and it shows any flash message on which the page will feed back to the admin like an error or success message. Form Forms The form is based on Flask-WTF that includes CSRF protection and form validation with form.hidden_tag() and form object. It also has a back to dashboard button so that one can easily access the dashboard.

Upon the submit of the form by the admin, the backend will look at existence of a similar named subject in the database. In case it does then a mistake message appears thereby blocking duplicates. In case the subject is new, then a Subject object is created and inserted into the database session, which is then permanently saved. The success message is shown through the Flask flash functionalities, and the dashboard shows the user the success message and redirects him/her to the

dashboard. This will make sure that one has only unique topics being added and gives instant feedback to the admin on the operation.

1.2.15 Enroll Student Page

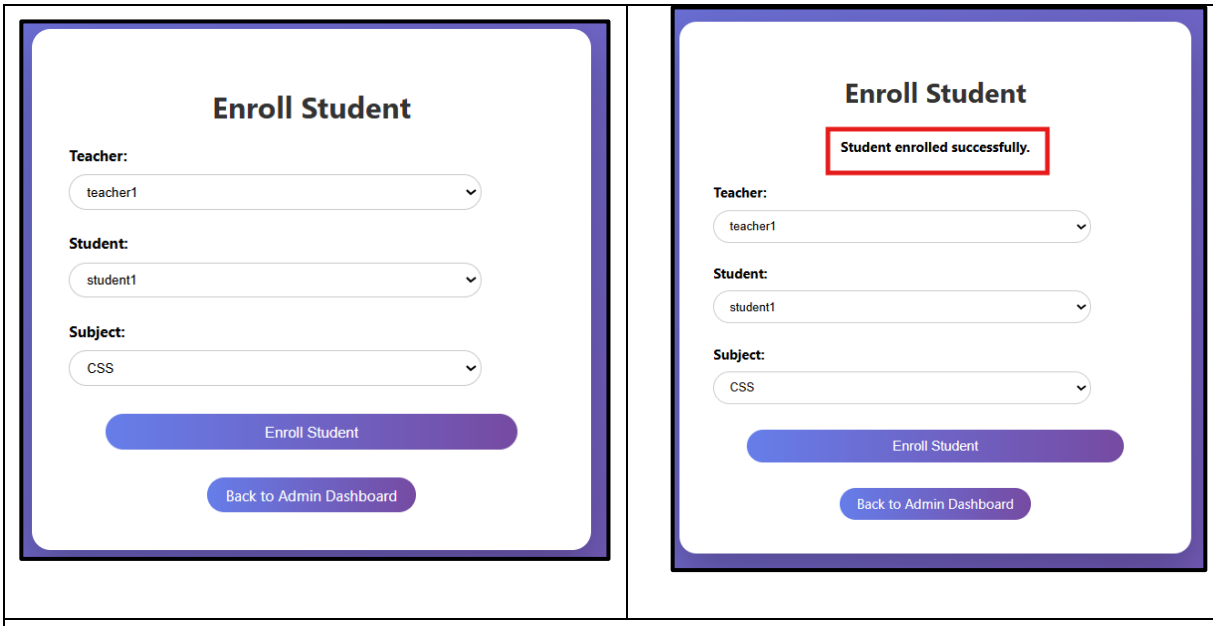


Figure 15 Enroll Student

Enroll Student page enables an administrator to give students a given subject under a certain teacher. Upon accessing this page by the admin, the system is first used to verify that an individual is an admin and redirects the rest to the login page. The page has an appearance of a form with three dropdowns: the first one is to choose a teacher, the second one is to choose a student, and the third one is to choose a subject. These drop downs are dynamically filled with data that is stored in the database hence the valid teachers, students and subjects can only be chosen by the admin. Flash messages are displayed at the top to give immediate feedback of any action i.e. error or successful enrollments. There is also a link to go to Admin Dashboard.

After the admin provides the form, the backend will firstly retrieve the selected teacher, student and subject IDs. Then it verifies with the TeacherStudentEnrollment table whether that student is already enrolled to that subject by the same teacher. In case of a duplicate enrolment, an error message is flashed to give warning to the admin. Otherwise, a new record of enrolment is made, inserted into the database and committed. There is a flash of a success message where the student is confirmed that he or she is enrolled. Such an arrangement will be necessary to make sure that

no student will be able to be enrolled more than once in a subject-teacher pair, and will give a clear feedback on whether the enrolment has been made.

1.2.16 View All Subject Attendance

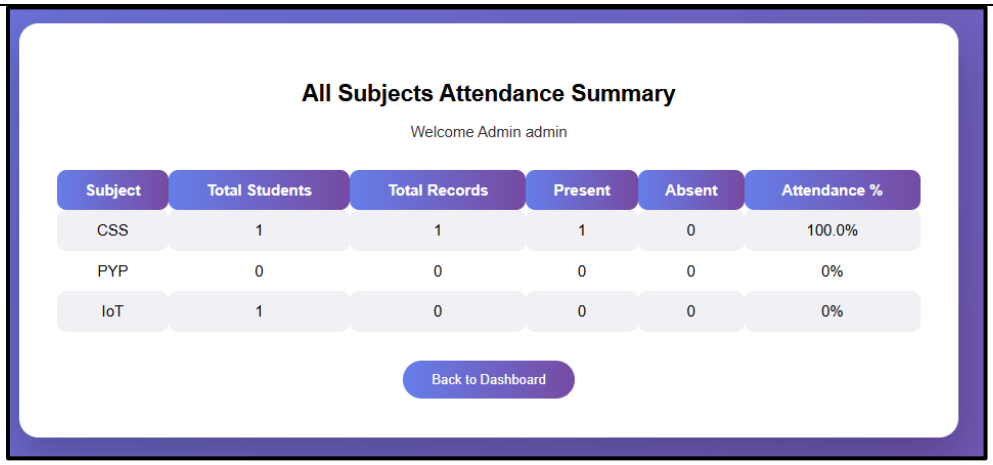


Figure 16 All Subject Attendance

The View All Attendance page displays the attendance statistics of all the subjects within the system, which is aimed at the admins. When an admin visits this page, the backend will first ensure that one has the role of the user before redirecting any other user to the login page. The page then retrieves all the subjects in the database and creates a summary on each subject. The key statistics mentioned in this summary are the total number of students enrolled to take the subject, the total number of attendance records taken, the amount of students who are marked as present, absent, and the overall attendance percentage. This information is presented in a table on the page so that it can be easily compared and every subject is represented in a row with the corresponding metrics being properly arranged.

At the backend, the system will search all subjects and query the TeacherStudentEnrollment table to find the number of students enrolled in the subject. It also views the Table of attendance in order to count all the records of the attendance, and the counts of Present and Absent statuses. The attendance rate is calculated as (total present/ total records) * 100 and rounded off to two decimal places. This summary is then submitted to the template which then displays it dynamically. In case there is no attendance data, there will be a message that records are not available. An option of a Back to Dashboard button will enable the admin to get back to the main dashboard in a short period

of time. This page helps the admins to keep track of the attendance trends and can quickly look at the attendance of a subject and see whether it is low or high.

1.2.17 Invalid Login

Figure 17 Wronged Password Entered

In the case of invalid login, that is, when one has entered the wrong password or a name that does not exist, the system flashes the user with a failure message. A counter in the database (failed attempts) is increased with each unsuccessful attempt. In case of an existing username with a wrong password, the counter is increased and the user is informed of the invalid password. In case the username does not exist at all, they are shown the message Username not found. The mechanism allows the user to know that they have failed to log in and the repeated attempts are also tracked to provide security.

1.2.18 Captcha Login Verification

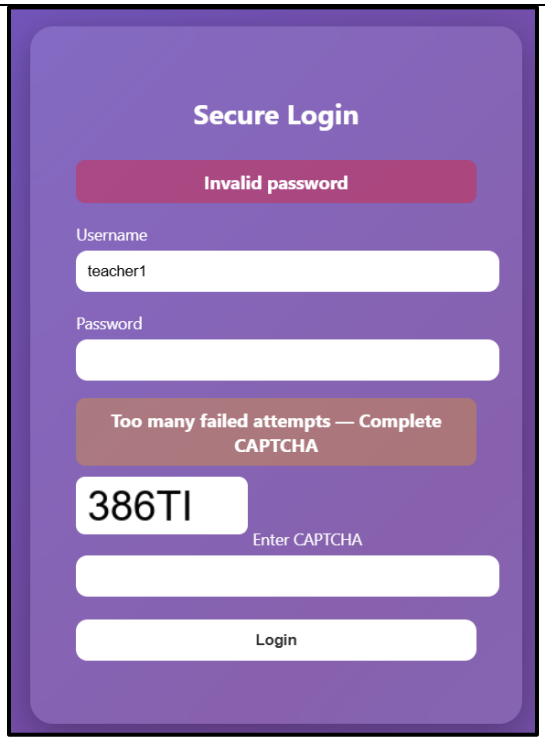


Figure 18 Captcha Verification

Once three unsuccessful attempts to enter the system have taken place, a CAPTCHA challenge is presented. It is shown right on the login page together with an dynamically generated image with random alpha-numeric characters, generated through captcha-util.py module. CAPTCHA text has to be entered correctly by the user or the entry will be declined, making it impossible to breach the passwords with the help of automated bots. In case the CAPTCHA input does not contain the same text as the generated one stored in the session, the system displays an error of Invalid CAPTCHA and prevents further log-in attempts till the correct CAPTCHA is typed.

1.2.19 Account Locked or Verification Via Email

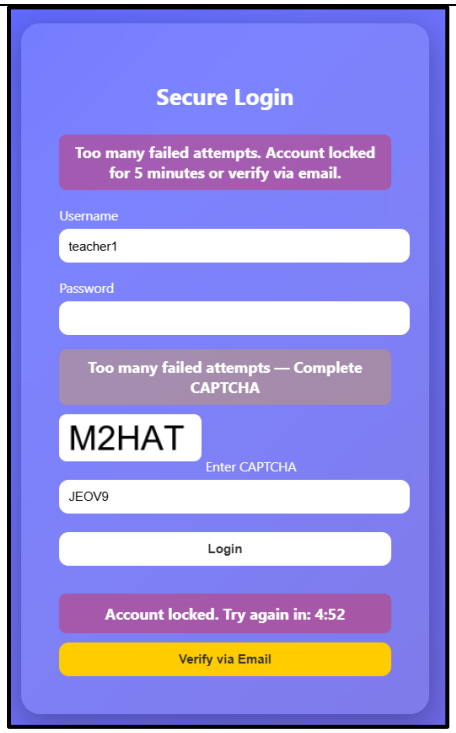
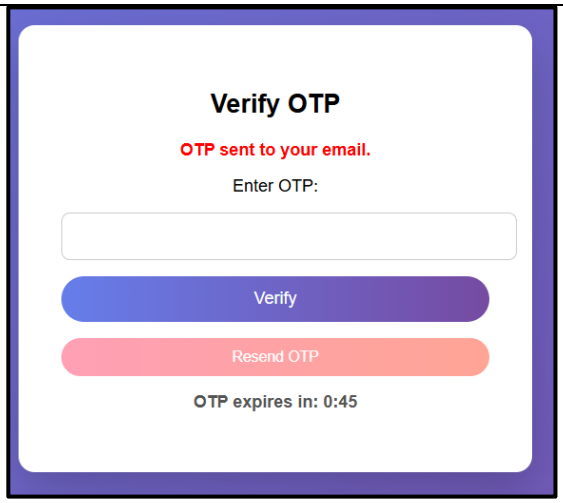


Figure 19 Account Locked

In case the user maintains the wrong set of credentials and makes five failures, the account is locked temporarily. In this lockout (default 5 minutes), the user is not allowed to log in even with the correct password. The system has a countdown timer of the remaining time at which the lock is attached. In order to get access back in time, the user offers an alternative option to verify his or her identity through OTP to his or her registered email.

1.2.20 Verification by OTP Page



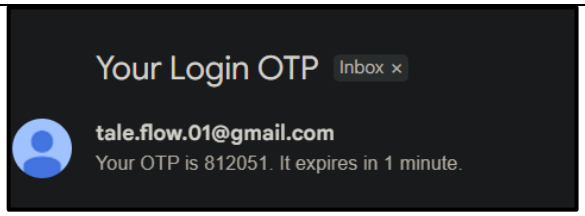
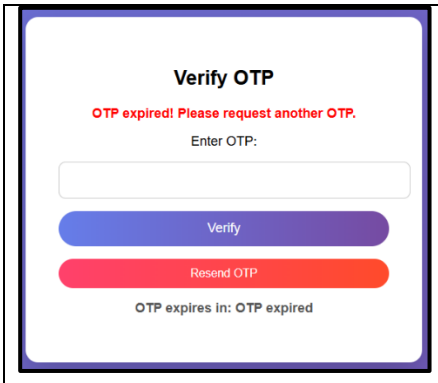


Figure 20 OTP Verification

Once a user chooses to verify the email after locking their account because of several unsuccessful attempts to log in, an OTP (One-Time Password) is created and sent to the email address of the registered user. This OTP is saved in session on a temporary server side in both cases with its expiration date (typically 1 minute). In the verify otp.html, the user is required to input the OTP in a special field. A countdown timer of the duration of the OTP is also indicated on the page. Upon entering the right OTP before expiry, the system will authenticate it and reassigns the failed attempt to log in and lock status of the user in the database and lets the user log in at once. This is so that despite locking the account to prevent intrusion, the actual user would be in a position to unlock the account without any form of insecurity.

1.2.21 OTP Expiration and Resend New OTP



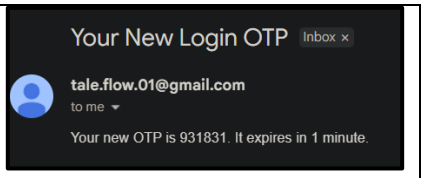


Figure 21 Resend OTP

In cases where the user remains in the page without entering the OTP and it goes out of time, the page shows a message that it has expired and the resend OTP button is made active. This button may be clicked by the user to obtain a new OTP. This system will then create a new OTP, change that session to use the new code and an expiration date and reposts it to the email of the user. The number of OTP resends is also limited (3 attempts) to avoid abuse. This flow also makes sure that the user will always have a valid OTP to prove the identity of the user and prevent the process by repeated or automated attempts.

1.2.22 OTP Verified and Account Unlocked

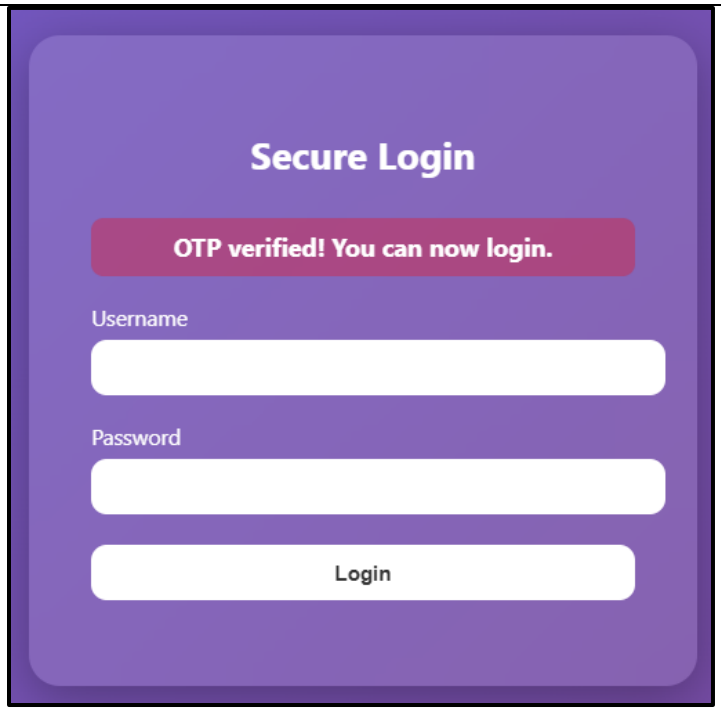


Figure 22 OTP Verified

They can unlock their account immediately when they are verified. This lockout period with OTP verification is resistant to brute force attacks and at the same time offers a recovery path to legitimate users.

1.3 Program Technical Backend

The Flask web framework in Python was also used to develop the backend of the Attendance Management System to process server-side logic, process request and communicate with the user interface and the database. The application is structured based on architecture in which routes are used to handle various system features including user authentication, dashboard access, attendance recording, and administrative management. The backend functions on the basis of the HTTP request which is transmitted by the frontend and executes the necessary operations and responds accordingly by rendering the HTML templates.

An object relational database was deployed through the use of SQLite and SQLAlchemy as the Object Relational Mapper(ORM). A number of database models were stipulated to classify the data structure of the system. User model contains user data such as their username, email, encrypted password and the access type e.g. administrator, teacher and student. Attendance model captures attendance information such as: student ID, teacher ID, subject ID, date and attendance status. Subject model keeps the records of the subjects that have been created by the administrator and TeacherStudentEnrollment model deals with the association between teachers, students, and subjects. These models enable the system to save and recall the attendance data effectively.

Authentication and session management is done through Flask-Login library, and this makes sure that only an authenticated user has access to the system and can only view a protected page of the system. The bcrypt hashing algorithm ensures that user credentials are stored safely without being accessed by an unauthorized user. Other security controls are applied to boost the security of logging in. As an illustration, unsuccessful logins are recorded and accounts are temporarily disabled upon numerous wrong passwords. To discourage the automated attack of a login attempt, a CAPTCHA verification system is also implemented following a number of unsuccessful attempts to log-in.

Moreover, there is an OTP-authentication system, which enables an account holder to use email verification to unlock his or her account in case it is locked. OTP is created randomly and email applications (Flask-Mail library) are used to send OTP to the registered email of the user. The OTP has a limited time of validity so as to enhance security. OTP values, expiration time and verification status are stored temporarily as session variables when performing the authentication process.

Role-based access control is provided in order to distinguish system capabilities to the administrator, teacher, and students. The administrators are given the functionality to create teacher accounts, student accounts, subjects, and enroll students to subjects and provide summaries on the attendance. Educators have permission to handle courses that have been delegated to them and take the attendance of registered students. Students can also see their attendance record and keep track of the attendance rate in each subject.

Password security measures are also incorporated in the backend system where the user must change his/her temporary passwords in the initial login. Authentication of password strength is done to ensure that the new passwords are complex enough by meeting the required complexity requirements as length, use of both upper and lower cases, numerals, and other special characters. The combination of these backend mechanisms ensures that there are secure authentication, data management, and appropriate control of the system access of the Attendance Management System.

2.0 Security Issue Identification

2.1 Unauthorized Access to Restricted Pages

```python
@app.route('/admin_dashboard')
@login_required
def admin_dashboard():
    if current_user.role != "admin":
        return redirect(url_for("login"))
```

Figure 23 Role-Based Access Control

Role-based access control follows to avert the restriction system pages to unidentified access. There are various user roles in the Attendance Management System including administrator, teacher and student. The system evaluates the role of the logged-in user with the current_user.role condition, and before the administrator dashboard is granted access, the system verifies the condition. When the user is not in the position of an administrator, the system redirects the user to the login page. This will make sure that only authorized users will be able to access sensitive administrative functions.

2.2 Brute Force Login Attempts

```python
# Lock after 5 attempts
if user.failed_attempts >= 5:
    user.lock_until = datetime.utcnow() + timedelta(minutes=5)
    remaining_seconds = 5 * 60
    flash("Too many failed attempts. Account locked for 5 minutes or verify via email.")
    session['locked_user'] = user.username  # store username for OTP verification
```

Figure 24 Brute Force Login Protection

This code has established defense against brute force entry attacks. A brute force attack is when an attacker repeatedly tries various password variations so as to have an unauthorized access to a user account. The failed attempts variable is used to record the number of failed_attempts to log in this system. Once the failed attempts reach five the account is temporarily locked in five minutes with

the help of the lock_until variable. This is done to ensure that the attackers do not keep on trying to make numerous attempts to log in and also enhances the general security of the system.

2.3 Automated Login Attacks (Bot Attacks)

```
# Require captcha after 3 failed attempts
if user.failed_attempts >= 3:
    if form.captcha.data != session.get("captcha_text"):
        flash("Invalid CAPTCHA")
        return render_template("login.html", form=form, remaining_seconds=remaining_seconds, failed_attempts=user.failed_attempt
```

Figure 25 CAPTCHA Protection Against Automated Attacks

The system has CAPTCHA verification to deter automated log-in attacks. In a few minutes, automated scripts or bots can present thousands of login requests. CAPTCHA verification is also to occur when there are three failed attempts to log in the system. The value typed into the CAPTCHA field is sent against the stored value in the session to confirm that the person is attempting to log in using their human fingers. This system will inhibit automated attacks on the login system.

2.4 Weak Password Usage

```
def is_password_valid(password):
    # Check all requirements
    if (len(password) >= 8 and
        re.search(r"[A-Z]", password) and
        re.search(r"[a-z]", password) and
        re.search(r"[0-9]", password) and
        re.search(r"[!@#$%^&*(),.?\":{}|<>]", password)):
        return True
    return False
```

Figure 26 Password Strength Validation

The validation of password strength is also applied so that the users can create secure passwords. Attackers can easily crack weak passwords by dictionary attack or brute force. This system has password validation rules that are set to ensure that stronger passwords are created. The password

should have eight or more characters that should include capital letters, lower letters, numbers, and special characters. The requirements make the password more complex and enhance the comprehensive security of user accounts.

2.5 Account Recovery Security Risk

```
# Generate new OTP
otp = str(random.randint(100000, 999999))
otp_expiration = datetime.utcnow() + timedelta(minutes=1)
```

Figure 27 OTP Generation

It will use an OTP (One-Time Password) system to increase the security of the account recovery process. In cases where the user account is locked because of several unsuccessful attempts at logging in, an OTP of 6 digits is generated. The OTP is sent to the email address used by the user and is valid within a vast duration of time. The expiration period makes sure that OTP is not reused beyond the stipulated time limit and this increases the level of security in the authentication process.

```
if entered_otp == otp:
    flash("OTP verified! You can now login.")

    session.pop("otp")
    session.pop("otp_expiration")
    session.pop("resend_count", None)

    username = session.get("locked_user")
    user = User.query.filter_by(username=username).first()

    if user:
        user.failed_attempts = 0
        user.lock_until = None
        db.session.commit()
```

Figure 28 OTP Verification

OTP verification is done to verify who the user is when recovering the account. The user enters an OTP which the system compares to the OTP generated in the session. When the OTP is a match, then it gets verified by the system and the stored OTP values are erased in the session. Following the successful verification, the unsuccessful attempts of logging in by the user are cleared and the

account lock is lifted. This system will make sure that the authorized user remains the only person that will get the locked account back.

2.6 Password Storage Risk

```
hashed_pw = bcrypt.hashpw(temp_password.encode("utf-8"), bcrypt.gensalt())
```

```
if bcrypt.checkpw(form.password.data.encode("utf-8"), user.password):
```

Figure 29 Secure Password Storage

The storage of secure passwords is done through the bcrypt hash algorithm. The passwords are not saved in plain text format but a hashed value of the password is made in the database. Bcrypt algorithm incurs salt and rounds of hashing and as such, it can be challenging to crack the original password even in the case of a database breach. The password that is entered is verified against the stored hashed password during the process of logging in with the help of the bcrypt verification function.

3.0 Requirement Identification

3.1 Secure Password Hashing Using bcrypt

```
hashed_pw = bcrypt.hashpw(temp_password.encode("utf-8"), bcrypt.gensalt())
```

```
if bcrypt.checkpw(form.password.data.encode("utf-8"), user.password):
```

Figure 30 Secure Password Hashing Using bcrypt

The system has the password hashing mechanism that is used to make sure that user passwords are stored safely. Rather than the passwords being stored in plain text, they are converted into a secure hash using the bcrypt hash algorithm and then they are stored as a hashed value in the database. At the time of the login, the password typed in is checked against the stored hashed password with bcrypt. This necessity will make the information retrieved by hackers impossible to get access to the original user passwords even when the database is compromised.

3.2 Login Attempt Limitation to Prevent Brute Force Attacks

```
if user.failed_attempts >= 5:
    user.lock_until = datetime.utcnow() + timedelta(minutes=5)
    remaining_seconds = 5 * 60
    flash("Too many failed attempts. Account locked for 5 minutes or verify via email."
```

Figure 31 Login Attempt Limitation to Prevent Brute Force Attacks

The system has a login attempt restriction system that prevents the occurrence of brute force attacks. The Brute force attacks are attacks initiated by attackers who mostly use various password combinations in their attempts to get unauthorized entry into user accounts. The attribute of failed_attempts is used to track the number of failed login attempts in the system. The number of failed attempts are used to block the user account temporarily with the help of the lock_until attribute when the number of unsuccessful attempts meets a specific limit. This system disables recurring attempts to log into the system and minimizes the chances of false entry.

3.3 CAPTCHA Verification to Prevent Automated Attacks

```
if user.failed_attempts >= 3:
    if form.captcha.data != session.get("captcha_text"):
        flash("Invalid CAPTCHA")
```

Figure 32 CAPTCHA Verification to Prevent Automated Attacks

CAPTCHA check has been used to ensure that automated bots are not used to repeat a login attempt. Large volumes of login requests can be sent out by the automated scripts within a few seconds. CAPTCHA verification is implemented in the system following a number of unsuccessful attempts at logging in. The user should type in the CAPTCHA code rightly to proceed with their login process. This is necessary so that the identification of the user performing the logins is not done by automated programs but by human beings.

3.4 Role-Based Access Control

```
@app.route('/admin_dashboard')
@login_required
def admin_dashboard():
    if current_user.role != "admin":
        return redirect(url_for("login"))
```

Figure 33 Role-Based Access Control

Role-based access control is done; in a way that system functions can only be accessed by authorized users. The Objects in the Attendance Management System include the administrator, teacher and student user roles. All the roles have various permissions in the system. The system checks the role of the logged-in user in conditional checks before access to a particular page is granted. In case the user lacks the necessary role, he/she is denied access to the page and redirected to the login page. This is necessary to ensure sensitive functions of the system are kept to unauthorized users.

3.5 OTP-Based Account Recovery System

```
otp = str(random.randint(100000, 999999))
otp_expiration = datetime.utcnow() + timedelta(minutes=1)

session['otp'] = otp
session['otp_expiration'] = otp_expiration.timestamp()
```

Figure 34 OTP-Based Account Recovery System

A mechanism of verification is done by OTP as a way of offering a secure account recovery mechanism. In case the user account is locked because of several unsuccessful attempts to enter the password, an OTP is created and emailed to the email address registered by the user. OTP has got a short duration of validity. To unlock the account, the user has to provide the right OTP so as to verify their identity. This is mandatory so that one can only regain access to an account using their legitimate user.

3.6 Password Strength Validation

```
def is_password_valid(password):
    # Check all requirements
    if (len(password) >= 8 and
        re.search(r"[A-Z]", password) and
        re.search(r"[a-z]", password) and
        re.search(r"[0-9]", password) and
        re.search(r"[!@#$%^&*(),.?\":{}|<>]", password)):
        return True
    return False
```

Figure 35 Password Strength Validation

The password strength validation check is introduced to make the users come up with good passwords in updating their account details. Dictionary attack or brute force can easily be used to crack weak passwords. The system has password validation rules that are established to root password creation. The password should have at least minimum length and must have both upper case letters and lower case letters, numbers and special characters. These are password

requirements that make the password more complex and enhance the overall security of the system.

4.0 Conclusion

To sum up, a web application based on the Attendance Management System was efficiently developed to facilitate and automatize the student attendance management process. The system had been implemented on a web-based architecture wherein the various user roles such as the administrators, teachers and students were supported. The core features, including user authentication, student enrollment, attendance recording, and account management, were also incorporated to make sure that the attendance data may be effectively handled. Attendance records are recorded and processed in a well-organized way through the application of a structured backend and database system, therefore, authorized users can easily access and manage information.

Moreover, it had various security measures that were executed to improve the stability and safety of the system. Role-based access control, OTP-based account recovery, account login monitoring, and validation of password strength were also added to secure user accounts and sensitive data. Such security practices are able to minimize the threat of unauthorized access and brute force attacks. On the whole, the created system proves the fact that an effective and safe web application can enhance attendance control and keep data integrity and accountability of users under control.

5.0 References

- OWASP. (2017). *Authentication · OWASP Cheat Sheet Series*. Owasp.org. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- Guide, D. (2025). *Implement Digital Identity - OWASP Developer Guide*. Owasp.org. https://devguide.owasp.org/en/04-design/02-web-app-checklist/06-digital-identity/
- Huang, S. (2022). Exploratory Analysis of Password and Login Security Methods. *Exploratory Analysis of P Y Analysis of Password and Login Security Methods D and Login Security Methods*, *18*. https://digitalcommons.odu.edu/cgi/viewcontent.cgi?params=/context/covacci-undergraduateresearch/article/1018/&path_info=Sofia_Huang_Exploratory_Analysis_of_Password_and_Login_Security_Methods.pdf
- JUN KIM, S., & MUN LEE, B. (2026). *IEEE xplore full-text PDF:* Ieee.org. https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=10759630
- Goud Kothinti, K. (2025). Mitigating one-time passcode (OTP) fraud: Strengthening authentication against emerging threats. *World Journal of Advanced Research and Reviews*, *26*(1), 1368–1378. https://doi.org/10.30574/wjarr.2025.26.1.1181
- Nawrocki, M., & Kołodziej, J. (2024). Vulnerabilities of Web Applications: Good Practices and New Trends. *Applied Cybersecurity & Internet Governance*. https://doi.org/10.60097/acig/199521
- Paul, A., Sharma, V., & Olukoya, O. (2024). SQL injection attack: Detection, prioritization & prevention. *Journal of Information Security and Applications*, *85*, 103871–103871. https://doi.org/10.1016/j.jisa.2024.103871
- Ablahd, A., & Dawwod, S. (2020). Using flask for SQLIA detection and protection. *Tikrit Journal of Engineering Sciences*, *27*(2), 1–14. https://doi.org/10.25130/tjes.27.2.01